Deriving and Implementing Gradient-Based Learning, From Linear Regression to a One-Hidden-Layer Neural Network

Prepared by: Amar Asfaw

1. Introduction

This report shows how I derived and implemented gradient-based learning first for Linear Regression and then for a neural network with one hidden layer. The main goal was to connect the chain rule to the actual parameter updates used during training. I used NumPy for the calculations and Matplotlib for the figures; no machine-learning framework, automatic differentiation, or built-in optimizer was used.

2. Dataset and Models

The dataset contains 100 float64 inputs evenly spaced from -2 to 2. The targets were generated from $Y = X^2 + \varepsilon$, with Gaussian noise of standard deviation 0.15 and dataset seed 42. Both X and Y have shape (100, 1), and every update uses all 100 samples.

Linear Regression uses one weight and one bias, both initialized to zero. The neural network has architecture 1 -> 8 -> 1: one input, eight ReLU neurons in one hidden layer, and one linear output. Its weights use seed 7 and scale 0.5, while the biases start at zero.

3. Linear Regression Gradient Derivation

For sample i, the model predicts a value using the current weight and bias. The overall training loss is the mean of the squared errors across all n samples:

$$\hat{y}_i = wx_i + b$$
$$L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

The derivative of the loss first passes through the prediction. The local derivatives are:

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{n}(\hat{y}_i - y_i), \qquad \frac{\partial \hat{y}_i}{\partial w} = x_i, \qquad \frac{\partial \hat{y}_i}{\partial b} = 1$$

Applying the chain rule and collecting the contribution from every sample gives the two gradients below. Gradient Descent then subtracts the learning-rate-scaled gradients from the current parameters:

$$\frac{\partial L}{\partial w} = \frac{2}{n}X^T(\hat{Y} - Y), \qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$
$$w \leftarrow w - \eta\frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta\frac{\partial L}{\partial b}$$

4. Neural-Network Forward and Backward Pass

The forward pass first maps each input to eight hidden values, applies ReLU, and then combines the hidden activations into one prediction:

$$Z_1 = XW_1 + b_1, \qquad A_1 = \mathrm{ReLU}(Z_1), \qquad \hat{Y} = A_1W_2 + b_2$$

Backpropagation starts from the MSE derivative, dY_hat = (2/n)(Y_hat - Y), and works backward. For an output-layer weight and a first-layer weight, the chain rule can be written component by component as:

$$\frac{\partial L}{\partial W_{2,j}} = \sum_i \frac{\partial L}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W_{2,j}} = \sum_i d\hat{y}_i A_{1,ij}$$
$$\frac{\partial L}{\partial W_{1,j}} = \sum_i \frac{\partial L}{\partial Z_{1,ij}}\frac{\partial Z_{1,ij}}{\partial W_{1,j}} = \sum_i dZ_{1,ij}x_i$$

The complete matrix backward pass is:

$$d\hat{Y} = \frac{2}{n}(\hat{Y} - Y), \qquad dW_2 = A_1^T d\hat{Y}, \qquad db_2 = \sum_i d\hat{Y}_i$$

$$dA_1 = d\hat{Y} W_2^T, \qquad dZ_1 = dA_1 \odot \mathrm{ReLU}'(Z_1)$$

$$dW_1 = X^T dZ_1, \qquad db_1 = \sum_i dZ_{1,i}$$

The transposes are not added only to make multiplication possible; they align the samples and features so that contributions from all samples are summed into gradients with the same shapes as the parameters. The implementation uses a ReLU derivative of zero when Z1 is exactly zero.

5. Backpropagation and Reverse-Mode Differentiation

Backpropagation computes the gradients; Gradient Descent uses those gradients to update the parameters. The forward pass stores Z1, A1, and the predictions. Starting from the single scalar loss, the backward pass reuses intermediate gradients to calculate dW2, db2, dW1, and db1.

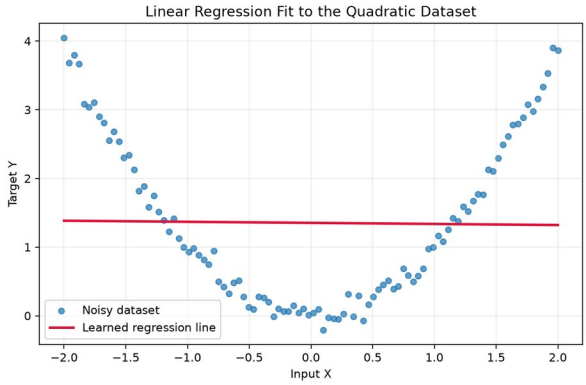
This is reverse-mode differentiation. It is efficient here because there is one final loss but many trainable parameters. One backward traversal provides all parameter gradients, while forward mode would need to propagate many separate parameter directions.

6. Implementation and Results

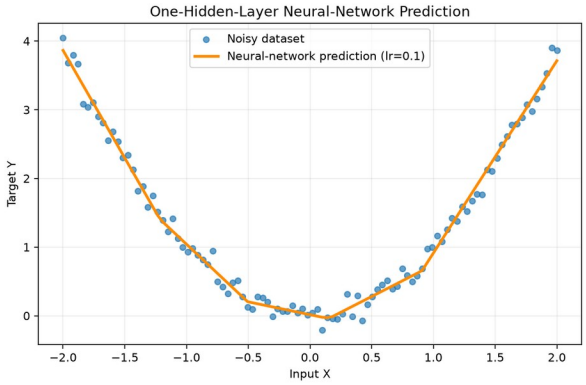
Linear Regression was trained with learning rate 0.05 for 1,000 updates. Its MSE decreased from 3.267617878211 to 1.437405676723. The learned weight was -0.015732313764 and the learned bias was 1.352728918547. The model therefore learned an almost horizontal line, which is the best straight-line approximation it could produce for this U-shaped dataset.

Learning rate	Initial MSE	Final MSE	Finite	Observed behavior
0.001	3.140907059	0.085072238	Yes	stable but slow
0.01	3.140907059	0.023692448	Yes	stable and faster
0.1	3.140907059	0.013474010	Yes	stable and fastest

All three neural-network runs started from the same initial parameters and completed 5,000 finite updates. The prediction from learning rate 0.1 followed the nonlinear target much more closely than the Linear Regression line.



Linear Regression fit



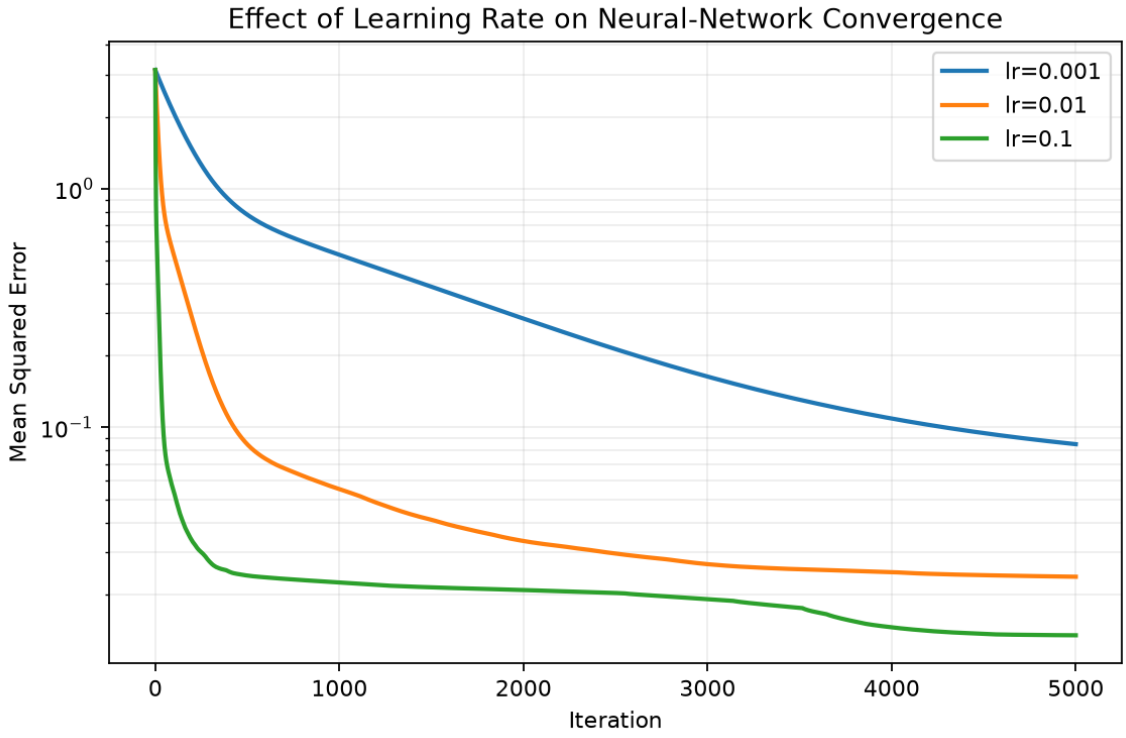
Neural-network prediction (lr = 0.1)

These plots show the main difference between the models: the linear model reduced its loss but remained limited to one straight line, while the ReLU network formed several linear regions that closely followed the U-shaped pattern.

7. Learning-Rate Comparison

The learning rate controlled how large each parameter update was. A rate of 0.001 remained stable but learned slowly. A rate of 0.01 reduced the loss faster, while 0.1 reduced it most quickly and reached the lowest final MSE, 0.013474010362. Therefore, 0.1 was the best among the three tested settings, not necessarily the best learning rate for every possible experiment.

The comparison graph uses a logarithmic MSE axis because the losses span a large range. This makes the three curves readable on the same graph without changing the underlying values.



Neural-network loss for the three tested learning rates

8. Challenges

The main practical difficulty was keeping every matrix two-dimensional and making each gradient match the shape of its parameter, especially the bias gradients. I also had to make sure that each learning-rate run started from the same original parameters and that iteration 0 was recorded before the first update. These details were important for a fair and reproducible comparison.

9. Conclusion

This project connected the chain rule directly to a working training algorithm. Linear Regression converged, but one straight line could not represent the quadratic pattern. The one-hidden-layer ReLU network had enough flexibility to follow the U-shape and produced a much lower MSE. Among the three tested learning rates, 0.1 converged fastest and ended with the lowest final loss.